

RBAC, ABAC & Authorization Models

Authorization Models Compared

Model	Grants based on	Strength
DAC	Ownership (file owner)	Worst – owner controls
MAC	System-enforced labels (Top Secret vs Secret)	Strong – user can't override
RBAC	Roles (admin, finance, user)	Common, manageable
ABAC	Attributes (dept, clearance, location, time)	Flexible, fine-grained

RBAC in Practice

Roles group PERMISSIONS, users belong to roles:

Role: finance-analyst

└ read reports

└ export csv

└ run monthly-recap job

Users: {Bob, Carol} → finance-analyst

RBAC advantages: fewer rules, easy to explain, matches organization structure. Traps: **role explosion** (every variant needs a role), **privilege creep** (users accumulate roles over years), **shared roles** (both "read" and "delete" in one).

ABAC for Fine Control

ABAC decides using *attributes*:

```
(&(department=finance)(clearance>=2)(time=08:00-18:00)(location=office))
```

Location-aware (only from office network)

Time-aware (only during business hours)

Context-aware (MFA present)

ABAC is more expressive but harder to manage and audit.

IDOR: When Authorization Code Fails

IDOR (Insecure Direct Object Reference) – the app checks WHO you are but never checks WHETHER you may touch THAT object:

```
GET /invoice/12345      → you own it? OK
GET /invoice/12346      → do YOU own it? ← if the app forgets the check → LEAK
```

The fix isn't a better model – it's a **verify-the-owner-check** on every object: fetch the object and confirm the requester owns it, in the same transaction.

Authorization bugs are separate from authentication. Strong MFA doesn't save you from IDOR – the attacker is already "authenticated as you".

The Authorization Testing Habit (Blue Team)

Login as user A (normal), user B (privileged), and guest

For EVERY request: try A accessing B's objects

Check whether the server enforces ownership ON THE DATA, not on the URL

Test hidden/guessed IDs, batch endpoints, and toggles

[← MFA, SSO & Federation](#)

[Privilege Escalation & Lateral...](#)